

Day 1 Deliverables: Requirements & High-Level Design

1. Requirements Document

Problem Statement

Build a scalable real-time fleet and delivery management system that allows administrators to track vehicles live on a map, assign delivery orders, and monitor delivery progress using WebSockets and map-based visualization. The system must support high concurrency, low latency, and horizontal scalability.

Actors

Admin: Manages orders and monitors live vehicle locations.

Driver: Sends live GPS data and handles assigned deliveries.

System: Matches drivers to orders and broadcasts updates.

Functional Requirements

Admin can view live driver locations, create and assign orders, and track order status.

Drivers can log in, receive orders, and send GPS updates.

System pushes real-time updates and handles disconnections.

Non-Functional Requirements

Scalability: Thousands of drivers supported.

Latency: Updates under 200ms.

Availability: 99.9% uptime.

Security: JWT authentication and RBAC.

Observability: Logs and metrics.

Fault tolerance with automatic recovery.

2. High-Level Design (HLD)

Architecture Overview

Clients communicate with backend services using REST and WebSockets via an API Gateway (NGINX). Auth, Order, and Tracking services are stateless and scale horizontally. Redis is used for caching and pub/sub. MongoDB stores persistent data.

Core Services

Auth Service: Authentication and authorization.

Order Service: Order creation and assignment.

Tracking Service: Real-time WebSocket-based vehicle tracking.

Data Stores

MongoDB stores users, orders, vehicles, and history.

Redis stores active drivers, live locations, and handles pub/sub messaging.

Communication Flow

REST APIs handle login and order management.

WebSockets handle real-time location updates and broadcasts.

3. README Summary

Overview

A real-time fleet and delivery management system built with React, Node.js, WebSockets, Redis, and MongoDB, designed for scalability and low latency.

Architecture

Service-based architecture with REST for CRUD operations and WebSockets for real-time tracking. Redis enables fast access and message fan-out.

Tech Stack

Frontend: React, Redux Toolkit, Google Maps.

Backend: Node.js, Express, Socket.IO.

Infrastructure: Docker, Kubernetes, NGINX, CI/CD, Terraform.

Scalability Decisions

Stateless services, Redis pub/sub for WebSocket scaling, throttled GPS updates, Kubernetes auto-scaling.